MOUNT CARMEL COLLEGE

KARUKADOM, KOTHAMANGALAM 686691

NAAC Accredited With A Grade (CGPA 3.21)

(Affiliated to MG University, Kottayam)

PROJECT REPORT ON

SMART COOKING ASSISTANT

(Chefly)

Submitted in partial fulfillment of the requirement for the award of the Bachelor of Computer Applications Degree from Mahatma Gandhi University, Kottayam.

Submitted By

ALEENA RAJU

Department of Computer Science

Mount Carmel College

Karukadom, Kothamangalam

MOUNT CARMEL COLLEGE

KARUKADOM, KOTHAMANGALAM 686691

NAAC Accredited With A Grade (CGPA 3.21)

(Affiliated to MG University, Kottayam)

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled "**CHEFLY**" is a bonafide record of the work done by **ALEENA RAJU (Reg.no 230021082710)** of the 5th Semester for the partial fulfilment of the **Bachelor of Computer Applications Degree from Mahatma Gandhi University, Kottayam** during the academic session **2023-2026.**

Project Guide | **Head of the Department**

Ms. Anjali Motilal | **Ms Angel George**

..... | .....

Viva Voce Examination held on .....

Internal Examiner ..... **External Examiner .....**

ACKNOWLEDGEMENT

It is a great privilege to express my sincere gratitude to all the respected personalities who guided, inspired, and helped complete this project effectively.

First and foremost, I thank the "God Almighty" for showering his blessings without which all the effort would not have been successful.

I sincerely thank our principal, **Fr. Shaji Mangalath**, for guiding this endeavour and making it successful

I would like to express my deepest gratitude to Assistant Professor **Ms. Angel George**, Head of the Department of Computer Science Department, for initiating this project and for her valuable guidance and suggestions.

I also thank **Ms. Anjali Motilal** my project guide, for her immense support, timely suggestions, and technical guidance throughout the project.

I sincerely thank my colleagues and friends for their continuous motivation, insightful discussions, and moral support during this project's challenging phases.

TABLE OF CONTENTS

INTRODUCTION	6
SYSTEM ANALYSIS	8
EXISTING SYSTEM	9
PROPOSED SYSTEM	10
SOFTWARE REQUIREMENT SPECIFICATION	11
HARDWARE SPECIFICATION	13
FEASIBILITY ANALYSIS	14
ECONOMIC FEASIBILITY	15
TECHNICAL FEASIBILITY	15
OPERATIONAL FEASIBILITY	16
BEHAVIORAL FEASIBILITY	17
DATAFLOW DAIGRAM	18
SYSTEM DESIGN	25
INPUT DESIGN	26
MODULE DESIGN	27
DATABASE DESIGN	29
OUTPUT DESIGN	39
TESTING	40
SYSTEM TESTING	40
UNIT TESTING	41
VALIDATION TESTING	41

INTEGRATION TESTING 42

USER ACCEPTANCE TESTING 43

BETA TESTING 43

ALPHA TESTING 43

SYSTEM IMPLEMENTATION 43

SYSTEM MAINTENANCE 44

FUTURE ENHANCEMENT 44

CONCLUSION 45

SCREENSHOTS 46

CODE 48

INTRODUCTION

Chefly is a web-based Recipe Management System designed to provide a structured and efficient digital platform for managing culinary activities. The system aims to simplify recipe organization, ingredient tracking, and user interaction through an integrated and user-friendly interface. By combining essential kitchen management functionalities within a single platform, Chefly enhances convenience and operational efficiency for users.

In many households, managing recipes and tracking available ingredients can be challenging, especially when information is scattered across various sources. Chefly addresses this issue by offering a centralized system where users can access, organize, and manage recipes effectively. The platform enables users to browse recipes categorized by cuisine, cooking time, and difficulty level. Each recipe provides comprehensive details, including ingredient lists, step-by-step preparation instructions, and optional video references to ensure clarity and ease of execution.

A key feature of Chefly is its support for user-generated content. Registered users can create and submit their own recipes to the platform. To maintain quality standards and ensure reliability, all submitted recipes undergo an administrative review and approval process before publication. This structured validation mechanism ensures consistency and accuracy across the system.

The system also includes a My Kitchen module, which allows users to maintain a record of ingredients currently available in their household. This feature promotes effective resource utilization by enabling users to identify suitable recipes based on existing ingredients, thereby reducing unnecessary purchases and minimizing food wastage. Furthermore, the Shopping List module assists users in organizing and tracking ingredients that need to be purchased, improving grocery planning and overall kitchen management.

To enhance user engagement and accessibility, Chefly integrates an interactive chatbot feature that provides recipe recommendations and cooking assistance. This component improves user experience by offering immediate support and personalized guidance.

Overall, Chefly serves as a comprehensive and technology-driven solution for recipe management and kitchen organization. By integrating recipe exploration, ingredient tracking, administrative validation, and interactive assistance, the system contributes to a more organized, efficient, and user-centered culinary experience.

Observation: Studying user interaction with existing recipe platforms to identify usability issues and areas for improvement in ingredient and recipe management.

User Experience Analysis

Understanding how users interact with the Chefly platform is important. This includes assessing:

- Users' expectations for a smooth and intuitive web-based cooking experience.
- Essential functionalities such as browsing recipes, managing ingredients, and organizing shopping lists.
- A simple and user-friendly interface for users to submit recipes and for administrators to manage content efficiently.

Analysis Process

- **Study of the Existing System:**

Identify limitations in current recipe management solutions, such as lack of a centralized platform, difficulty in organizing ingredients, inefficient recipe approval processes, and non-intuitive user interfaces.

- **Design of the New System:**

Propose a web-based platform – Chefly – that enables users to browse and manage recipes efficiently, maintain ingredient records, and submit their own recipes for administrative approval.

- **Cost-Benefit Analysis:**

Compare the development and maintenance costs with the potential benefits, such as improved user convenience, better kitchen organization, enhanced user engagement, and the project's academic value.

- **Detailed Examination:**

View the system as a whole by identifying inputs (e.g., recipe submissions, ingredient entries, user queries) and outputs (approved recipes, updated kitchen records, chatbot responses) and tracing them through the system workflow.

Conclusion

System analysis for Chefly involves a comprehensive study of existing recipe management platforms, identifying their limitations, and proposing a technologically feasible, user-friendly, and scalable solution. By applying structured analysis, the system ensures an efficient, organized, and accessible platform for users to manage recipes and kitchen activities effectively.

SYSTEM ANALYSIS

System analysis is the process of breaking a problem into manageable components for detailed study. For the Chefly Recipe Management System, system analysis focuses on understanding the operations required to address challenges in recipe organization and ingredient management, define system boundaries, and determine the technical and functional requirements needed to develop a user-friendly and scalable solution.

Objectives of the Analysis

The primary objective of system analysis is to determine the feasibility and effectiveness of the proposed system. The system should:
Deliver optimal performance with minimal cost and development time.
Fulfill its core objective of providing an efficient and reliable recipe management experience.
Be implemented using modern and sustainable technologies within budget constraints.
Offer a significantly improved user experience compared to existing recipe management platforms.

Key Elements of the System Analysis for GreenHub

Feasibility Study
A feasibility study determines whether the Chefly project is economically, technologically, and operationally viable.

Economic Feasibility: Assessing the cost of development against the benefits, such as improved kitchen management, increased user engagement, potential growth in user base, and the overall academic value of the project.

Technological Feasibility: Ensuring that the proposed features — including recipe browsing, ingredient tracking, recipe submission with admin approval, and an interactive chatbot — can be effectively implemented using the chosen technologies such as Django, HTML, CSS, and a relational database system.

Operational Feasibility: Confirming that the system meets the needs of end-users (managing recipes and ingredients), contributors (submitting recipes), and administrators (reviewing and approving content while maintaining system quality).

EXISTING SYSTEM

Traditionally, the process of managing recipes and tracking ingredients is often handled through scattered sources such as cookbooks, websites, handwritten notes, or basic applications that lack integration and user-centered design. Many existing recipe management solutions are fragmented or limited in functionality, making it difficult for users to efficiently organize recipes, monitor available ingredients, and maintain a structured kitchen management system.

Key Characteristics of Existing Systems

- L**ack of a Unified Platform:** Most current recipe management solutions focus only on storing recipes or only on ingredient tracking, but not both, leading to poor integration and inefficiency.
- **Manual Organization Methods:** In many cases, users rely on handwritten notes, separate apps, or multiple websites to manage recipes and ingredients, resulting in disorganization and difficulty in tracking information.
- **Limited User Contribution Features:** Existing platforms often provide restricted options for users to submit and manage their own recipes, limiting content diversity and user engagement.
- **Absence of Structured Ingredient Tracking:** Users frequently lack a proper system to monitor available ingredients, which can lead to unnecessary purchases and food wastage.
- **Poor User Experience:** Many existing applications have non-intuitive interfaces or lack responsiveness across devices, making navigation complex and less user-friendly.

Limitations of the Existing System

- **Inefficient Recipe Organization:** The absence of a centralized digital system forces users to depend on scattered sources, making recipe management time-consuming and unstructured.
- **Limited User Contribution:** Complicated or restricted submission processes discourage users from sharing and managing their own recipes effectively.
- **No Structured Ingredient Monitoring:** Users often lack real-time tracking of available ingredients, leading to poor planning and unnecessary purchases.
- **Device Compatibility Issues:** Many existing applications are not fully responsive or optimized for different devices, reducing accessibility and convenience.
- **Fragmented Management Approach:** The lack of an integrated platform connecting recipe browsing, ingredient tracking, and administrative approval results in poor coordination and reduced user satisfaction.

PROPOSED SYSTEM

The proposed system, Chefly, is a fully functional, web-based Recipe Management System designed to address the limitations of existing recipe and kitchen management solutions. It enables users to browse, manage, and organize recipes efficiently while also allowing registered users to submit their own recipes to the platform. Once reviewed and approved by the administrator, these recipes become accessible to all users.

The system is developed using a modern and scalable technology stack — Django for backend development, HTML and CSS for the front-end interface, and a relational database for data management — ensuring a seamless, responsive, and user-friendly experience.

Key Features of the Proposed System

Multi-Device Responsive Design: Chefly is developed with a responsive design approach, ensuring consistent performance across desktops, tablets, and smartphones. Users can access recipes and manage kitchen activities anytime and from any device.

Dynamic and User-Centric Interface: The platform features a clean and intuitive interface that simplifies recipe browsing, ingredient management, and content submission. Smooth navigation enhances overall usability.

User Module: Registered users can browse recipes, manage their available ingredients through the My Kitchen feature, create shopping lists, and submit their own recipes. Users can also receive confirmation once their submitted recipes are approved.

Recipe Contributor Module: Users can submit new recipes to the platform, which are reviewed by the administrator before publication. This ensures quality control while encouraging community participation.

Admin Dashboard: Administrators manage the platform by reviewing and approving submitted recipes, monitoring user activities, and maintaining overall system integrity to ensure smooth and secure operation.

SOFTWARE REQUIREMENT SPECIFICATION

ABOUT THE FRONT-END PHP

The front-end of the Chefly system is developed using standard web technologies such as HTML (HyperText Markup Language) and CSS (Cascading Style Sheets). These technologies are responsible for designing and structuring the visual and interactive components of the web application. The front-end acts as the presentation layer of the system, allowing users to interact with the platform in a clear, organized, and efficient manner.

HTML is used to create the structural foundation of web pages, including headings, forms, buttons, tables, images, and navigation menus. It defines how content is arranged and displayed on the browser. CSS is used to enhance the visual appearance of the website by applying styles such as colors, fonts, spacing, alignment, and layout design. CSS also enables responsive design, ensuring that the interface adapts smoothly to different screen sizes and devices.

In the Chefly system, the front-end is carefully designed to provide a user-friendly and intuitive experience. Users can easily browse recipes, submit new recipes, manage ingredients through the My Kitchen module, and update their shopping lists. Well-structured forms are implemented to collect user input efficiently, while clear navigation elements improve accessibility and usability.

Responsive design principles are applied to ensure compatibility across desktops, tablets, and smartphones. This enhances accessibility and ensures that users can access the platform conveniently from various devices. The front-end interface is designed to minimize complexity, reduce user errors, and provide smooth interaction with backend processes.

Overall, the front-end of Chefly plays a vital role in delivering an engaging and accessible user experience. It serves as the bridge between users and the system's backend logic, ensuring seamless communication and effective functionality.

ABOUT THE BACK-END

Database Management

A database is a structured collection of information organized for efficient storage, retrieval, and management. In the Chefly system, the database plays a vital role in storing and managing essential information related to users, recipes, ingredients, shopping lists, and administrative actions.

Chefly utilizes a relational database management system to maintain data in a centralized and organized manner. All application data — including user registration details, recipe submissions, ingredient records, and approval statuses — is securely stored within the database. This centralized structure enables the system to perform core operations such as user authentication, recipe approval, ingredient tracking, and content management efficiently.

The backend is developed using Django, which interacts seamlessly with the database through its built-in Object-Relational Mapping (ORM) framework. The ORM simplifies database operations by allowing developers to interact with database records using Python code instead of complex SQL queries. This improves development efficiency, reduces errors, and enhances maintainability.

The backend ensures data integrity, security, and consistency throughout the system. It processes user requests, validates inputs, manages business logic, and generates appropriate responses that are displayed on the front-end interface. Additionally, administrative controls are implemented to monitor user activities, review recipe submissions, and maintain system reliability.

Overall, the backend of Chefly serves as the core processing unit of the system, handling data management, authentication, validation, and overall application logic to ensure smooth and secure platform functionality.

ABOUT THE OS

Windows Operating System

The operating system used for the development and execution of Chefly is Windows 11 (22H2), one of Microsoft's most advanced and widely used operating systems. Windows has played a crucial role in shaping the modern graphical user interface (GUI) experience and continues to be a preferred choice for developers and users worldwide.

Why Windows 11 (22H2) for GreenHub

User-Friendly Interface: Windows offers an intuitive and visually appealing interface, making it easier for developers to build, test, and manage web applications like Chefly.

Enhanced Performance and Stability: The Windows 11 22H2 version includes significant improvements in performance, memory management, and stability, providing a smooth and efficient development environment.

Plug and Play Connectivity: Its built-in support for plug-and-play devices allows quick connection and configuration of hardware components, improving the overall system setup and testing process.

Internet Connectivity: Easy and stable connectivity to the internet and local networks ensures seamless access to remote databases, online APIs, and web hosting environments, which are essential for Chefly's backend operations.

Widespread Compatibility: Most modern development tools, software frameworks, and web servers have optimized versions for Windows, ensuring that all technologies used in Chefly (PHP, MySQL, HTML, CSS) work flawlessly.

File and System Management Tools: The integrated File Explorer and system utilities make organizing project files, managing databases, and deploying web applications more efficient.

HARDWARE SPECIFICATION

Selecting the right hardware configuration is crucial for efficient software development, testing, and deployment. The system should have a powerful processor, sufficient memory, and storage to handle both the backend database operations and the frontend application smoothly.

CONFIGURATION OF HARDWARE

Processor: Intel 5 5600H

Speed: 3.3 GHz – 4.4 GHz (Turbo Boost)
RAM: 8 GB
Storage: 512 GB SSD (for faster data access and application performance)
Keyboard: Standard PS/2 Keyboard
Mouse: Standard Optical Mouse

FEASIBILITY ANALYSIS

Feasibility is defined as the practical extent to which a project can be successfully developed and implemented. For Chefly, a feasibility study was conducted to determine whether the proposed Smart Cooking Assistant platform is practical, workable, and beneficial for users and administrators.

The feasibility study considers:
Resource Availability: Ensuring the availability of necessary hardware, software tools, and skilled development personnel for successful implementation.
Cost Estimation: Evaluating the overall cost of development, deployment, hosting, and future maintenance of the system.
User Benefits: Assessing how the system enhances recipe management, ingredient tracking, and overall user convenience.
Maintenance Costs: Identifying ongoing expenses related to updates, database management, and technical support.

Objectives of the Feasibility Study
To analyze whether Chefly meets the requirements of users and administrators effectively.
To determine if the system can be developed using the chosen technologies within the specified budget and timeframe.
To assess whether the platform can integrate with additional tools or features if required.
To ensure the system is adaptable to future enhancements and evolving user requirements.

Types of Feasibility Checked
Technical Feasibility: Verifying that the selected technology stack and development tools can support all system functionalities, including recipe management, ingredient tracking, user submissions, and administrative controls.
Operational Feasibility: Ensuring that the system is user-friendly, meets functional requirements, and enhances overall recipe and kitchen management experience.
Economic Feasibility: Evaluating whether the development and maintenance costs are justified by the expected benefits, such as improved efficiency, better user engagement, and long-term academic value.

ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the development of Chefly is financially practical and delivers value relative to the costs incurred. It considers the expenses involved in system development, deployment, and maintenance, and compares them with the benefits the platform provides.

Development Costs: The cost of building Chefly is minimal since the technologies used, such as Django, HTML, CSS, and relational database systems, are open-source and freely available.

Feasibility Study Costs: Expenses related to requirements analysis, system design, and planning are low, as the project follows a clear structure and defined objectives.

Hardware and Software Costs: No additional hardware investment is required. Existing computer systems are sufficient for development, testing, and deployment.

Team and Training Costs: The system is developed by the project team, and users can be guided through simple documentation or instructions, minimizing training expenses.

Chefly is economically feasible because development costs are low, no new hardware is required, and the system provides significant benefits such as improved recipe management, better ingredient tracking, and enhanced user convenience. The overall impact on resources is minimal, making it a cost-effective and practical solution.

TECHNICAL FEASIBILITY

Technical feasibility evaluates whether the available hardware, software, and technological resources are sufficient to develop and operate the Chefly system efficiently within the allocated time and budget. It also examines the capability of the development team and the suitability of the selected technologies.

- Hardware and Software Resources: The existing hardware configuration (PCs with adequate processing power, 8GB RAM, and sufficient storage) and operating system are sufficient to develop, test, and deploy the Chefly platform effectively.
- Technology Stack:

    - Frontend: HTML and CSS are used for designing the user interface and implementing responsive layouts.
    - Backend: Django is used for server-side development and database interaction, ensuring secure and efficient data management.

These technologies are well-established, stable, and widely used in web development.

- **Development Team Skills:** The project team possesses the necessary expertise in Django, database management, HTML, and CSS, ensuring that all system functionalities can be implemented effectively.
- **Browser Compatibility:** Chefly is a web-based application accessible through modern web browsers, making it platform-independent and easily available to users.
- **Support and Community:** The technologies used are widely supported by large developer communities, allowing easy troubleshooting, regular updates, and future enhancements.

Based on the available resources, the expertise of the development team, and the selected technology stack, Chefly is technically feasible. The platform can be developed efficiently and is capable of meeting all functional requirements, including recipe management, ingredient tracking, user submissions, and administrative control, without significant technical limitations.

OPERATIONAL FEASIBILITY

Operational feasibility assesses how well the GreenHub platform can perform the required tasks to meet user and business needs. It focuses on whether the system will function correctly after development and whether it will be adopted effectively by users once deployed.

- **Priority of Requirements:** Ensures that all essential features, such as recipe management, ingredient tracking, and administrative approval, are implemented to meet the primary needs of users and administrators.
- **Acceptability of Solution:** Confirms that the system developed is practical, user-friendly, and aligned with the expectations of stakeholders and end-users.
- User Adaptability: Evaluates whether users can easily adapt to the system, considering interface design, navigation, and overall usability.
- **Client Satisfaction:** The system is developed based on defined requirements, with continuous evaluation and feedback during each stage to ensure objectives are achieved.
- **Compliance with Standards:** Chefly adheres to standard web development and database management practices, ensuring reliability, security, and maintainability.

Based on continuous stakeholder feedback, adherence to standard development practices, and a user-friendly design approach, Chefly is operationally feasible. The platform is expected to function efficiently, be widely accepted, and remain easy to use for both users and administrators.

BEHAVIORAL FEASIBILITY

Behavioral feasibility assesses how easily users and stakeholders will accept and adapt to the new system. People are often resistant to change, so it is important to ensure that the system reduces workload, simplifies processes, and encourages adoption.

- **Reduced Manual Effort:** In traditional recipe management methods, users rely on manual notes or scattered sources to organize recipes and track ingredients. Chefly automates these processes, reducing manual effort and minimizing errors.
- **Time Efficiency:** The platform enables users to quickly browse recipes, manage ingredients, and organize shopping lists online, saving time and improving convenience.
- **Ease of Adaptation:** The interface is simple and intuitive, allowing users and administrators to adopt the system without extensive training.
- **Optimized Resource Utilization:** By streamlining recipe and ingredient management, users can focus on efficient meal preparation and better kitchen organization, improving overall productivity.

Chefly is behaviorally feasible because it significantly simplifies recipe and ingredient management compared to traditional manual methods. By automating tasks such as recipe organization, ingredient tracking, and administrative approval, the system reduces repetitive manual work and minimizes the chances of errors.
The platform is designed with a clean, intuitive interface that ensures ease of navigation and understanding for users of different technical backgrounds. Since the system does not require complex training or advanced technical knowledge, both users and administrators can adopt it comfortably without resistance.
Furthermore, Chefly promotes a gradual and smooth transition from conventional methods, such as handwritten notes or scattered online resources, to a centralized and structured digital solution. Its simplicity, clarity, and efficiency encourage consistent usage, improve user confidence, and support long-term acceptance of the system for effective kitchen management.

DATA FLOW DIAGRAM (DFD)

Data Flow Diagram is a network that describes the flow of data and processes that change, or transform, data throughout the system. This network is constructed by use a set of symbols that do not imply a physical implementation. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a process, which transforms the data and creates, output-data-flows which go to other processes or external entities or files. Data in files may also flow to processes as inputs.

There are various symbols used in a DFD. Bubbles represent the processes. Named arrows indicate the data flow. External entities are represented by rectangles. Entities supplying data are known as sources and those that consume data are called sinks. Data are stored in a data store by a process in the system. Each component in a DFD is labelled with a descriptive name. Process names are further identified with a number.

The Data Flow Diagram shows the logical flow of a system and defines the boundaries of the system. For a candidate system, it describes the input (source), outputs (destination), database (files) and procedures (data flow), all in a format that meet the user's requirements.

The main merit of DFD is that it can provide an overview of system requirements, what data a system would process, what transformations of data are done, what files are used, and where the results flow.

This network is constructed by use a set of symbols that do not imply a physical implementation. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a process, which transforms the data and creates, output-data-flows which go to other processes or external entities or files. External entities are represented by rectangles. Entities supplying data are known as sources and those that consume data are called sinks. Data are stored in a data store by a process in the system. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a process, which transforms the data and creates, output-dataflows which go to other processes or external entities or files. Data in files may also flow to processes as inputs.

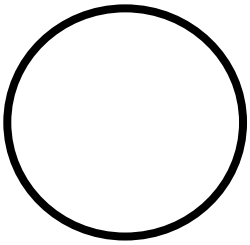
Rules for constructing a Data Flow Diagram

1. Arrows should not cross each other.
2. Squares, circles and files must bear names.
3. Decomposed data flow squares and circles can have same time.
4. Choose meaningful names for data flow.
5. Draw all data flows around the outside of the diagram.

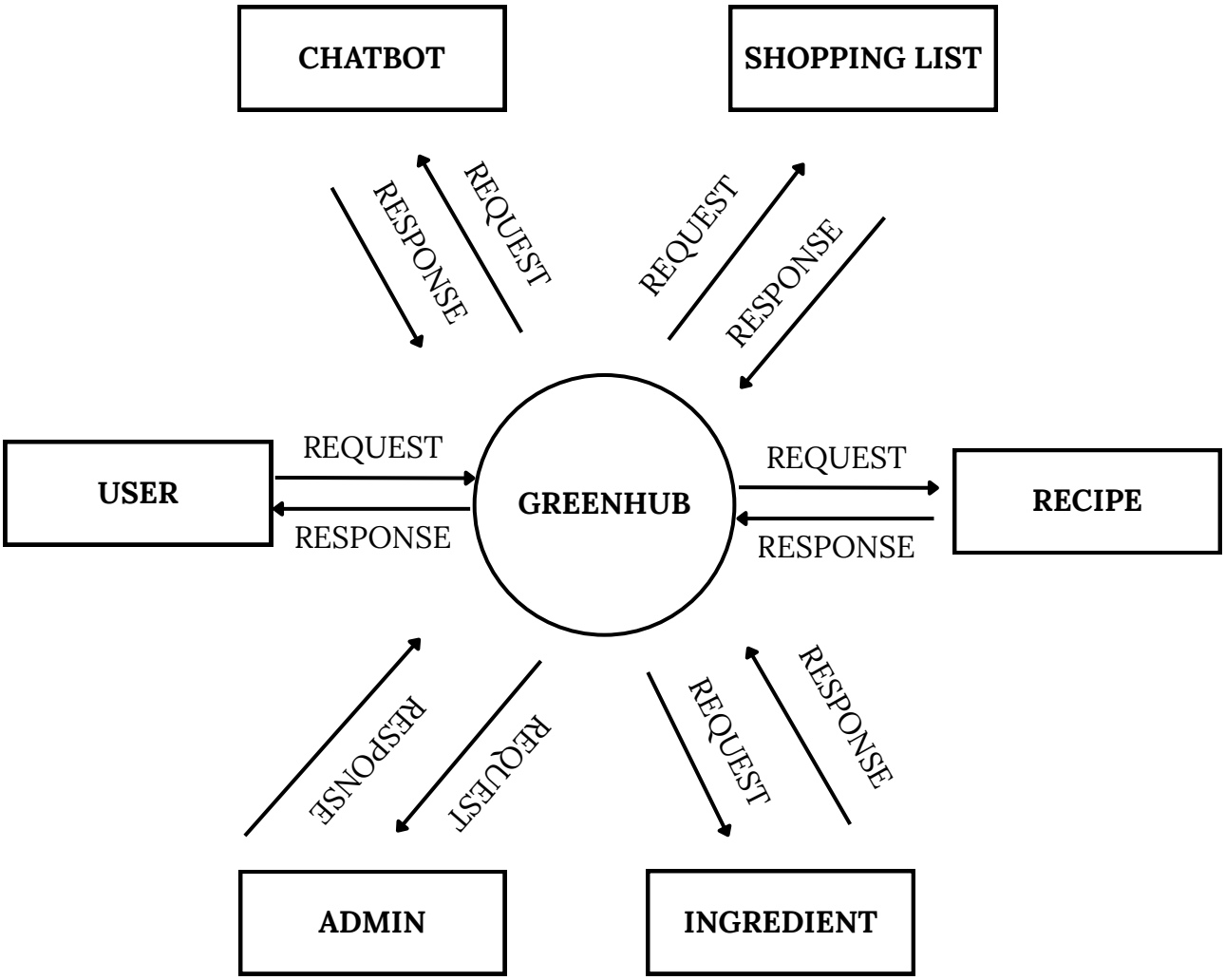
Data Flow Diagram: Each component in a DFD is labelled with a descriptive name. Process name are further identified with number. Context level DFD is draw first. Then the process is decomposed into several elementary levels and is represented in the order of importance. A DFD describes what data flow (logical) rather than how they are processed, so it does not depend on hardware, software and data structure or file organization. A DFD methodology is quite effective; especially when the required design.

Basic Data Flow Diagram Symbols

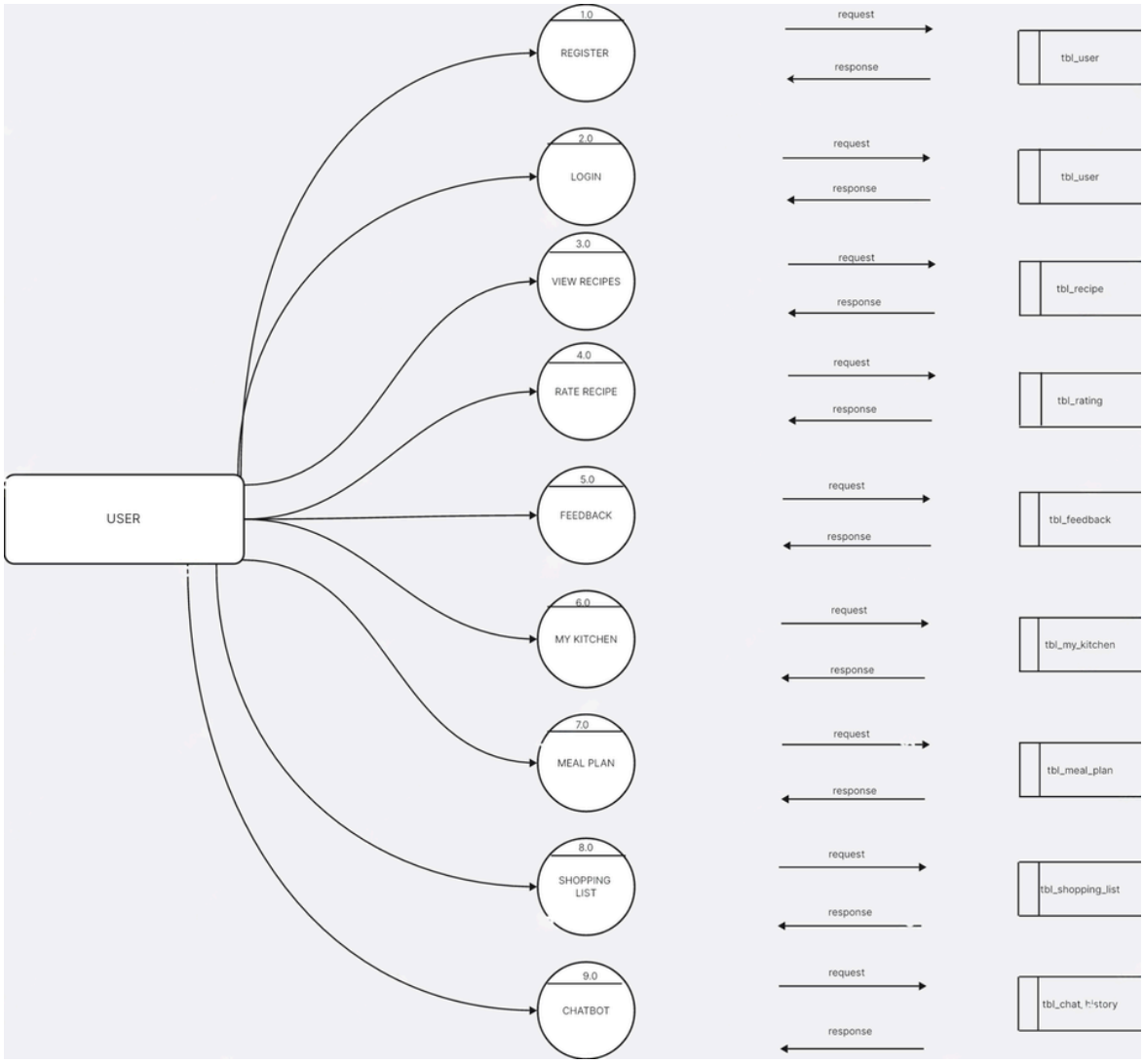
Data Flow Diagram Symbols

	A data flow is a route, which enables packets of data to travel from one point to another. Data may flow from a source to a process and from data store or process. An arrow line depicts the flow, with arrow head pointing in the direction of the flow.
	Circles stands for process that converts data in to information. A process represents transformation where incoming data flows are changed into outgoing data flows.
	A data store is a repository of data that is to be stored for use by a one or more process may be as simple as buffer or queue or sophisticated as relational database. They should have clear names. If a process merely uses the content of store and does not alter it, the arrowhead goes only from the store to the process. If a process alters the details in the store then a doubleheaded arrow is used.
	A source or sink is a person or part of an organization, which enters or receives information from the system, but is considered to be outside the contest of data flow model.

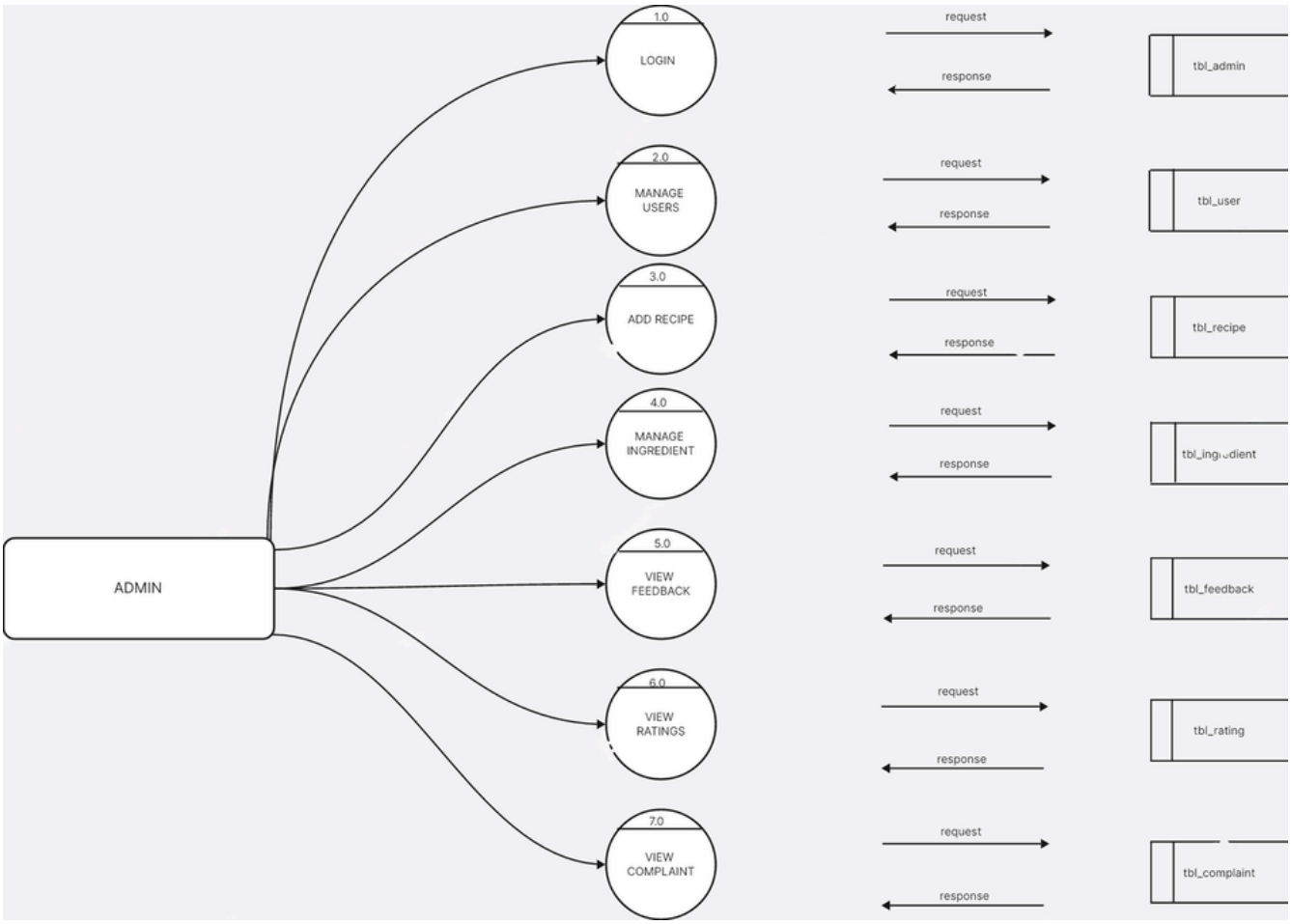
CONTEXT LEVEL



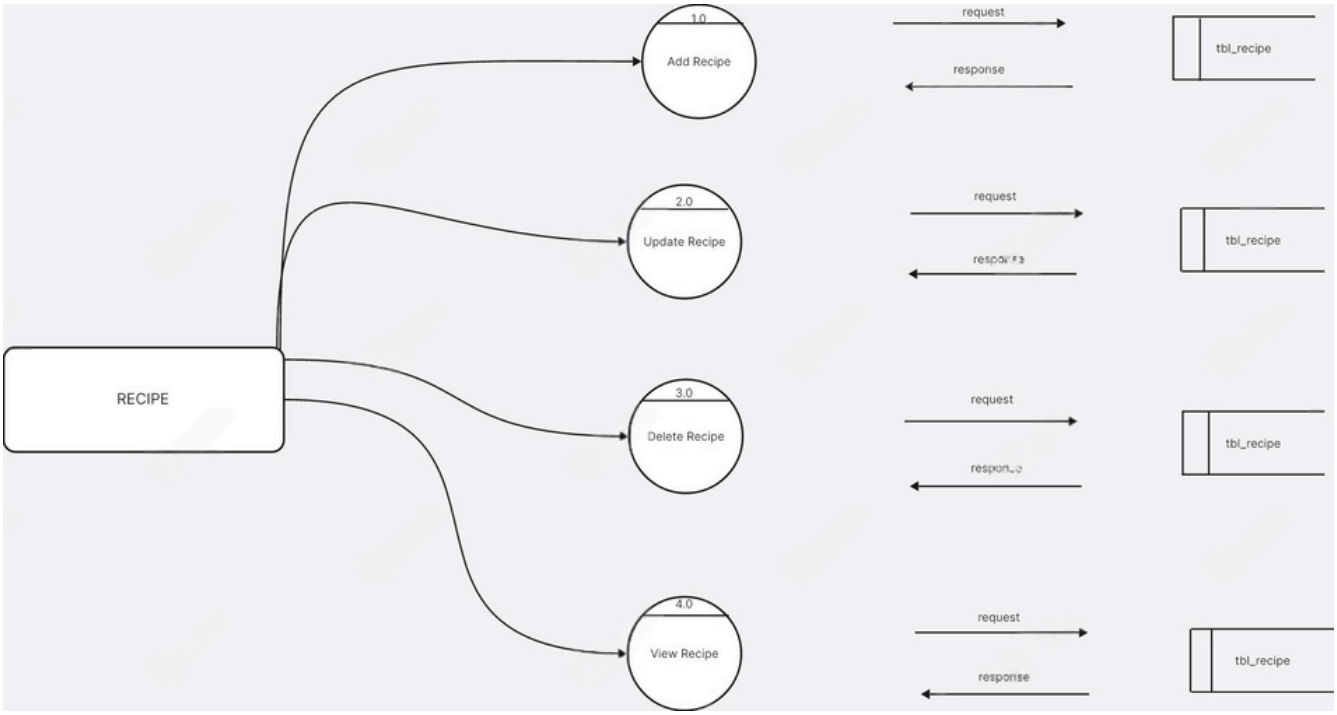
LEVEL 1 : USER


USER
1.0
REGISTER
2.0
LOGIN
3.0
VIEW RECIPES
4.0
RATE RECIPE
5.0
FEEDBACK
6.0
MY KITCHEN
7.0
MEAL PLAN
8.0
SHOPPING LIST
9.0
CHATBOT
request
response
tbl_user
tbl_recipe
tbl_rating
tbl_feedback
tbl_my_kitchen
tbl_meal_plan
tbl_shopping_list
tbl_chat_history

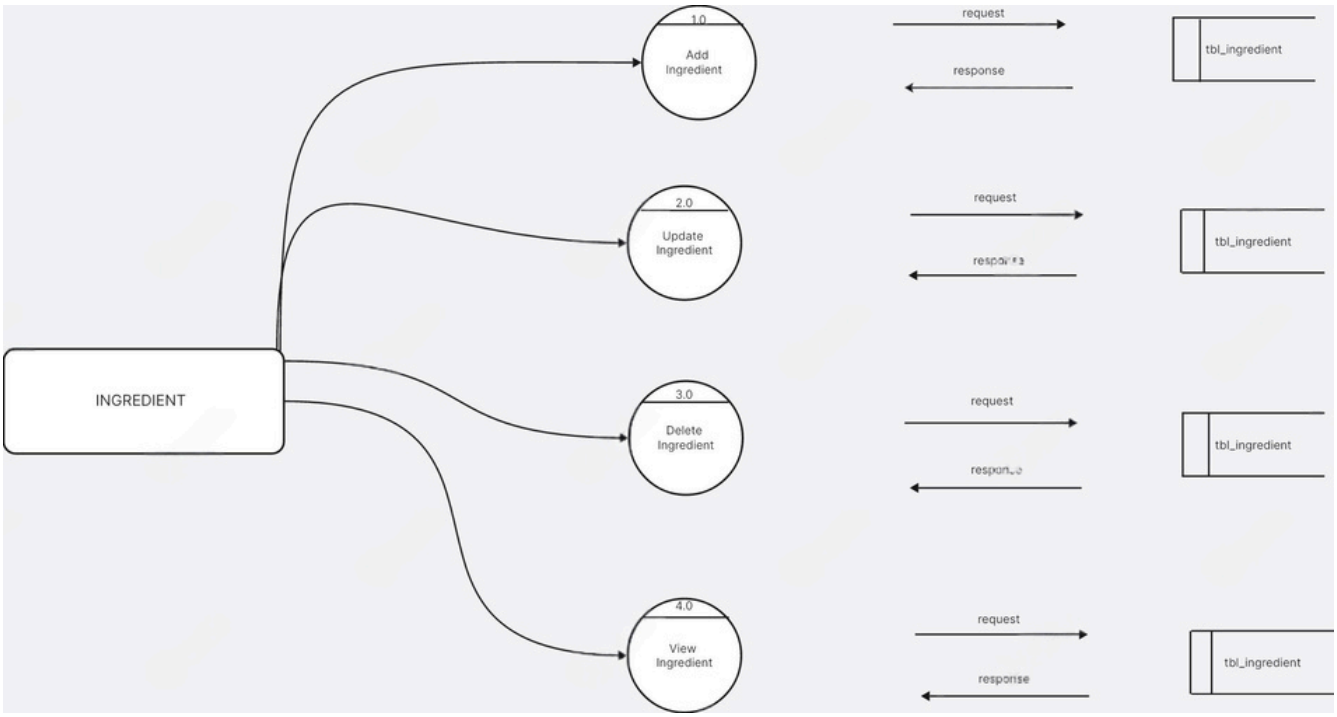
LEVEL 1 : ADMIN



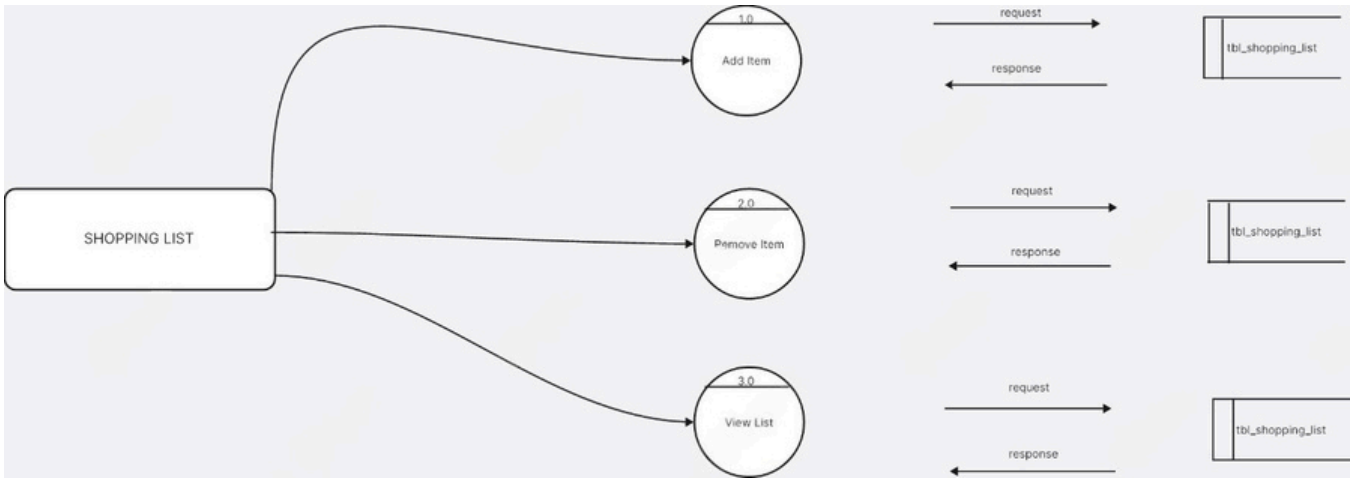
LEVEL 1 : RECIPE



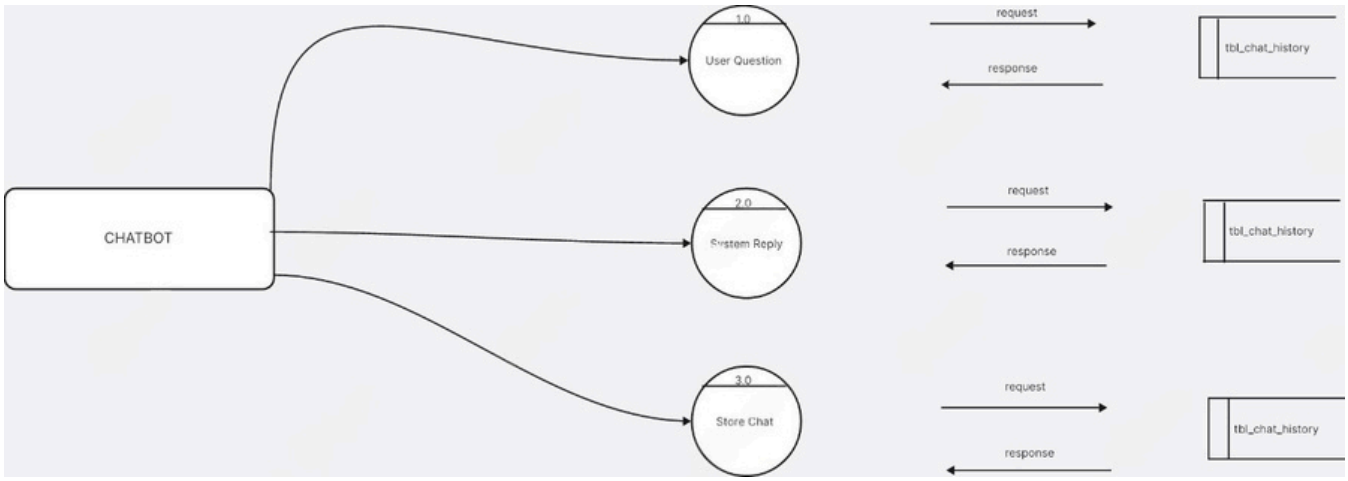
LEVEL 1 : INGREDIENT



LEVEL 1 : SHOPPING LIST



LEVEL 1 : CHATBOT



SYSTEM DESIGN

System design is the conceptual framework that defines the structure, behavior, and multiple views of a system. An architecture description is a formal representation of the system, organized to support understanding and analysis of its structure. It identifies system components, their externally visible properties, and the relationships and interactions between them. This description serves as a blueprint for development, ensuring that all parts work together effectively. Formal methods used to describe such architectures are known as Architecture Description Languages (ADLs).

System architecture can be viewed as a collection of representations of an existing or proposed system. It communicates the components of the system, the relationships among them, and the governing rules. These components may include hardware, software, documentation, facilities, procedures, or organizational roles. Architecture primarily focuses on internal interfaces between subsystems and the interaction between the system and its external environment, particularly users.

A well-defined structural design reduces complexity, supports easier modification, and enables parallel development of different system components. Procedural design translates the structural architecture into detailed descriptions of software processes. Architectural design treats the system structure as a key functional requirement, ensuring clarity, scalability, and maintainability.

The system follows a three-tier architecture:

1. Presentation Layer (User Interface): Handles user interaction. It displays controls, collects user input, and performs basic validation.
2. Business Layer (Business Logic): Contains the core application logic where data processing and decision-making occur.
3. Data Layer (Database Layer): Manages data storage and retrieval. It is responsible for maintaining and updating information in the database.

In a three-tier architecture, communication occurs sequentially between layers. Each layer interacts only with its adjacent layer, ensuring better organization, security, and maintainability of the system.

- Update station details and manage available slots.
- Monitor booking requests and manage scheduling.
- View user feedback and ratings for their station.
- Communicate with users in case of maintenance or slot changes.

INPUT DESIGN

Input design plays a critical role in the development of the Online Library Management System, ensuring seamless interaction between the user and the system. The input design includes converting user-oriented inputs into a computer-based format and establishing methods that are efficient, accurate, and user-friendly. Key features of the input design for this system include:

- **User Interface Design:** The platform will feature a graphical user interface (GUI) that allows users to interact with the system intuitively. The interface design considers user friendliness, consistent formatting, and interactive dialogue to guide users effectively Input Methods: Users can input data using structured and interactive forms. These forms are designed with validation messages to ensure data accuracy and minimize errors. For example, users can add items to their cart, specify delivery addresses, and make payment details seamlessly.
- **Error Validation:** Input fields include validation mechanisms to ensure only accurate and complete data is submitted. For instance, fields for quantities, delivery addresses, and payment information will be verified before being processed.
- **Interactive Features:** Drop-down menus, auto-fill suggestions, and real-time error checks enhance the input experience, making it straightforward and efficient for users to provide the required information.

Steps in Input Design for GreenHub:

- Review Input Requirements: Identify the essential data the system needs, such as user registration details, vehicle information, slot booking details, and charging station registration data.
- Input Data Flow: Define how user-provided data flows from input forms (like booking forms or station registration) through the processing stages to the database and backend logic.
- Source Documents: Determine the primary sources where user inputs are collected, such as user signup forms, slot booking pages, and charging station registration forms.

MODULE DESIGN

Modular programming is a software design approach that divides a complex system into smaller, independent, and manageable components called modules. Each module performs a specific function and interacts with other modules through defined interfaces. This approach reduces complexity, improves maintainability, and supports scalability.
In the Chefly Recipe Management System, the system is divided into the following primary modules:

1. User Module

- User registration and login
- Profile management
- Save favorite recipes
- Manage kitchen ingredients
- Meal planning
- View health and nutrition details
- Provide feedback

2. Recipe Module

- Uploading recipes
- Categorizing recipes
- Viewing recipe details
- Searching recipes
- Filtering recipes
- Managing recipe content

3. Ingredient Module

- Adding ingredients
- Updating ingredient details
- Categorizing ingredients
- Removing ingredients
- Searching ingredients
- Managing ingredient inventory

4. Shopping List Module

- Adding ingredients to shopping list
- Removing ingredients from shopping list
- Updating shopping list
- Viewing shopping list
- Exporting shopping list

5. Admin Module

- Uploading recipes and ingredients
- Categorizing data
- Managing users
- Managing online platform
- Monitoring system activity
- Ensuring platform security

6. Chatbot Module

- Answering user queries
- Suggesting recipes
- Recommending videos
- Providing cooking tips
- Finding nearby grocery stores

Importance of Input Design

Data Accuracy: Reliable input methods ensure that accurate and validated data enters the system — such as user details, recipes, ingredient quantities, and shopping list items — leading to efficient processing and dependable results.
Efficiency: Intuitive and well-structured input interfaces reduce the time and effort required from users when adding recipes, ingredients, or shopping items, improving overall usability.

User Interaction: A well-designed input interface enables smooth and efficient interaction, allowing users to register, submit recipes, manage ingredients, or create shopping lists without confusion.

DATABASE DESIGN

A database is a structured collection of interrelated data stored with minimal redundancy to allow multiple users to access information quickly and efficiently. The primary goal of a database is to make information retrieval easy, fast, cost-effective, and flexible. In Chefly, the database integrates all user, recipe, ingredient, shopping list, and feedback information to ensure smooth operation and consistent data management.

Database design is a critical part of any management information system and is essential for ensuring data accuracy, integrity, and security. In designing the Chefly database, several objectives are considered:

- **Ease of Learning and Use**: The database should be simple to interact with for both the system and its administrators.
- **Controlled Redundancy:** Data is stored efficiently to avoid unnecessary duplication.
- **Data Independence:** Changes in physical storage or structure do not affect application programs.
- **Cost-Effective Access to Information:** Enables retrieval of extensive information without high overhead.
- **Accuracy and Integrity:** Ensures correct and consistent data throughout the system.
- **Recovery from Failure:** Maintains system reliability through backup and recovery mechanisms.
- Privacy and Security: Protects sensitive user data and access controls.

- **Performance:** Provides fast and reliable access to data for multiple simultaneous users.

In Chefly, a Relational Database Management System (RDBMS) such as MySQL is used. The key distinction between an RDBMS and other types of DBMS is the separation between logical data (how the program views data) and physical data (how it is actually stored). This separation ensures better data management, flexibility, and scalability while maintaining centralized control over all system information.

Normalization

Designing a database is a critical task, and normalization is an essential technique that ensures the database is efficient, consistent, and scalable. Normalization is the process of organizing data in a database to reduce redundancy, avoid anomalies, and ensure that the database can grow without requiring extensive changes to applications or data structures.

In Chefly, as new recipes, ingredients, and user data are added, the database must handle updates, deletions, and additions efficiently. Without proper design, a database may face several problems:

Repetition of Information: Storing duplicate data unnecessarily.
Inability to Represent Certain Information: Difficulty in accurately modeling relationships.
Loss of Information: Errors during updates, deletions, or insertions.

Normalization minimizes these issues by organizing data into relations that follow certain rules called normal forms. In this process, no information is lost, so it is also referred to as "no-loss decomposition."

First Normal Form (1NF): A relation is in 1NF if all its attributes are atomic, meaning each field contains only a single value from its domain. The objective is to remove repeating groups and ensure that each entry in the table contains only one value per attribute.

Second Normal Form (2NF): A table is in 2NF if it is already in 1NF and every non-key attribute is fully functionally dependent on the entire primary key, not just a part of it. This eliminates partial dependencies.

Third Normal Form (3NF): A table is in 3NF if it is in 2NF and every non-key attribute depends only on the primary key, not on other non-key attributes. This removes transitive dependencies, further improving data integrity.

By applying normalization in Chefly, the database can efficiently handle growing numbers of recipes, ingredients, and user interactions, while minimizing maintenance costs, avoiding anomalies, and ensuring high-quality, reliable data management.

Table Structure

Table is a collection of complete details about a particular subject. These data are saved in rows and Columns. The data of each Row are different units. Hence, rows are called RECORDS and Columns of each row are called FIELDS.

Data is stored in tables, which is available in the backend the items and data, which are entered in the input, form id directly stored in this table using linking of database. We can link more than one table to input forms. We can collect the details from the different tables to display on the output.

There are 14 tables used in this project :

1. AdminProfile
2. User
3. UserProfile
4. Recipe
5. Ingredient
6. RecipeIngredient
7. UserRecipe
8. ShoppingList
9. MyKitchen
10. LikedRecipe
11. Feedback
12. FeedbackLike
13. Complaint
14. ChatBot

2.tbl_user

Description: This table stores the details of users who register in the Chefly system.

Field	Data Type	Description
id	Integer (PK)	Unique ID for each user
user_name	Varchar	Unique ID for each user
user_email	Varchar	Email address of the user
user_password	Varchar	Password for user login
user_profile_image	Image	Profile image of the user
user_datetime	DateTime	Date and time when the user registered

2. tbl_admin_profile

Description: This table stores administrator details used to manage the Chefly system.

Field	Data Type	Description
id	Integer (PK)	Unique ID for each administrator
admin_name	Varchar	Name of the administrator
admin_email	Varchar	Email address of the administrator

3. tbl_admin_profile

Description: This table stores additional profile information of users.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
image	Image	Profile picture of the user
bio	Text	Short biography or description of the user

4. tbl_recipe

Description: This table stores details of recipes available in the Chefly system.

Field	Data Type	Description
id	Integer (PK)	Unique ID for each recipe
recipe_name	Varchar	Name of the recipe
recipe_description	Text	Description of the recipe
recipe_difficulty	Varchar	Difficulty level of the recipe
recipe_cooking_time	Integer	Cooking time in minutes
recipe_image	Image	Image of the recipe
recipe_steps	Text	Step-by-step instructions for preparing the recipe
recipe_video_link	URL	Video tutorial link for the recipe

5. tbl_ingredient

Description: This table stores ingredients used in recipes along with category and nutrition information.

Field	Data Type	Description
id	Integer (PK)	Unique ID for each ingredient
ingredient_name	Varchar	Name of the ingredient
ingredient_category	Varchar	Category of the ingredient
ingredient_image	Image	Image of the ingredient
nutrition_info	Text	Nutritional information of the ingredient
is_in_shopping_list	Boolean	Indicates whether the ingredient is added to shopping list
is_bought	Boolean	Indicates whether the ingredient is purchased

6. tbl_recipe_ingredient

Description: This table connects recipes with ingredients and stores the quantity required for each ingredient.

Field	Data Type	Description
id	Integer (PK)	Unique ID
recipe_id	Foreign Key	Reference to tbl_recipe
ingredient_id	Foreign Key	Reference to tbl_ingredient
quantity	Varchar	Quantity of ingredient required

7. tbl_user_recipe

Description: This table stores recipes submitted by users which require admin approval.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
recipe_name	Varchar	Name of the user submitted recipe
recipe_image	Image	Image of the recipe
recipe_description	Text	Description of the recipe
recipe_ingredients	Text	Ingredients used in the recipe
recipe_steps	Text	Preparation steps
recipe_video_link	URL	Video tutorial link
recipe_cooking_time	Integer	Cooking time in minutes
recipe_difficulty	Varchar	Difficulty level of the recipe
status	Varchar	Status of the recipe (Pending/Approved/Rejected)
created_at	DateTime	Date and time when recipe was submitted
updated_at	DateTime	Date and time when recipe was updated

8. tbl_shopping_list

Description: This table stores ingredients that users add to their shopping list.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
ingredient_id	Foreign Key	Reference to tbl_ingredient
shoppinglist_quantity	Integer	Quantity of ingredient to purchase
shoppinglist_purchaasedatetime	DateTime	Date and time when item was added
shoppinglist_status	Boolean	Indicates whether the item is purchased
shoppinglist_external_link	URL	External link to buy the ingredient

9. tbl_mykitchen

Description: This table stores ingredients currently available in the user's kitchen

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
ingredient_id	Foreign Key	Reference to tbl_ingredient
mykitchen_quantity	Varchar	Quantity available in the kitchen
mykitchen_addeddatetime	DateTime	Date and time when ingredient was added

10.tbl_liked_recipe

Description: This table stores recipes liked by users.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
recipe_id	Foreign Key	Reference to tbl_recipe
liked_datetime	DateTime	Date and time when recipe was liked

11.tbl_feedback

Description: This table stores feedback messages and ratings given by users.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
message	Text	Feedback message submitted by user
rating	Integer	Rating provided by the user
created_at	DateTime	Date and time when feedback was submitted

12. tbl_feedback_like

Description: This table stores likes given by users to feedback messages.

Field	Data Type	Description
id	Integer (PK)	Unique ID for each administrator
user_id	Foreign Key	Reference to tbl_user
feedback_id	Foreign Key	Reference to tbl_feedback

13.tbl_complaint

Description: This table stores complaints submitted by users and replies provided by administrators.

Field	Data Type	Description
id	Integer (PK)	Unique ID
user_id	Foreign Key	Reference to tbl_user
message	Text	Complaint message submitted by the user
created_at	DateTime	Date and time when complaint was submitted
reply	Text	Reply given by administrator
status	Varchar	Status of complaint (Pending or Resolved)

14. tbl_chatbot

Description: This table stores user queries and chatbot responses in the Chefly system.

Field	Data Type	Description
id	Field	Unique ID
user_id	Foreign Key	Reference to tbl_user
message	Text	Message sent by the user
response	Text	Reply generated by the chatbot
created_at	DateTime	Date and time when the chat occurred

OUTPUT DESIGNS

Output design is very important concept in the computerized system, without reliable output the user may feel the entire system unnecessary and avoids using it.The proper output design isimportant in any system and facilitates effective decision making. The output design of this system includes various reports. output requirements are designed during system analysis. An application is successful only when it can provide efficient and effective reports.

The goal of the output design is to capture the output and get the data into a format suitable for the computer .it is very helpful to produce the clear, accurate and speedy information for end users. A major form of the output is the harder copy from the pointer and screen reports. Printouts are designed around the output requirements of the user. Allowing the user to view the sample screen is important because the user is the ultimate judge of the quality of output. Output of this project is provided in the form of reports created using crystal report tool.

TESTING

Coding conventions are a set of guidelines for a specific programming language that recommend programming style, practices and methods for each aspect of a piece program written in this language. These conventions usually cover file organization, indentation, comments, declarations, statements, white space, naming conventions, programming practices, programming principles, programming rules of thumb, architectural best practices, etc. These are guidelines for software structural quality.Software programmers are highly recommended to follow these guidelines to help improve the readability of their source code and make software maintenance easier. Testing is the process of correcting a program with intends of finding an error. Different types of testing are :

- System Testing
- Unit Testing
- Integrated Testing
- Black Box Testing
- White Box Testing
- Validation Testing
- User Acceptance Testing

SYSTEM TESTING

The objective of system testing is to ensure that all individual programs are working as expected, that the programs link together to meet the requirements specified and to ensure that the computer system and the associated clerical and other procedures work together. The initial phase of system testing is the responsibility of the analyst who determines what conditions are to be tested, generates test data, produced a schedule of expected results, runs the tests and compares the computer produced results with the expected results with the expected results. The analyst may also be involved in procedures testing. When the analyst is satisfied that the system is working properly, he hands it over to the users for testing. The importance of system testing by the user must be stressed. Ultimately it is the user must verify the system and give the go ahead.

The objective of system testing is to ensure that all individual programs are working as expected, that the programs link together to meet the requirements specified and to ensure that the computer system and the associated clerical and other procedures work together. The initial phase of system testing is the responsibility of the analyst who determines what conditions are to be tested, generates test data, produced a schedule of expected results, runs the tests and compares the computer produced results with the expected results with the expected results. The analyst may also be involved in procedures testing. When the analyst is satisfied that the system is working properly, he hands it over to the users for testing. The importance of system testing by the user must be stressed. Ultimately it is the user must verify the system and give the go ahead.

UNIT TESTING

In computer programming, unit testing is a method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures are tested to determine if they are fit for use. In this testing we test each module individual and integrated the overall system. Unit testing focuses verification efforts on the smaller unit of software design in the module. This is also known as module testing. The modules of the system are tested separately. The testing carried out during programming stage itself. In this testing step each module is found to working satisfactory as regard to the expected output from the module. There are some validation checks for verifying the data input given by the user which both the formal and validity of the entered.

It is very easy to find error debug the system.
I have continued Unit Testing from the starting of the coding phase itself. Whenever I completed one small sub module, some amount of testing was done based on the requirements to see if the functionality is aligned to the gathered requirements.

VALIDATION TESTING

At the culmination of Black Box testing, software is completely assembled as a package, interface errors have been uncovered and corrected and final series of software tests, Validation tests begins. Validation testing can be defined many was but a simple definition is that validation succeeds when the software functions in a manner that can be reasonably accepted by the customer. After validation testhas been conducted one of the two possible conditions exists.

1. The function or performance characteristics confirm to specification and are accepted.

2. A derivation from specification uncovered and a deficiency list is created.

3. I have given various validations in our forms so that there will be a neat format for the data's that are entered on to the website. I have also given an already existing validation so that the data redundancy is reduced; same data is not entered twice.

INTEGRATION TESTING

Integration testing (sometimes called integration and testing, abbreviated I&T) is the phase in software testing in which individual software modules are combined and tested as a group. Software components may be integrated in an iterative way or all together ("big bang"). Normally the former is considered a better practice since it allows interface issues to be located more quickly and fixed. Data can be lost across an interface; one module can have an adverse effort on the other sub functions when combined by, may not produce the desired major functions. Integrated testing is the systematic testing for constructing the uncover errors within the interface. This testing was done with sample data. The developed system has run success full for this sample data. The need for integrated test is to find the overall system performance

Integration testing is a logical extension of unit testing. In its simplest form, two units that have already been tested are combined into a component and the interface between them is tested. A component, in this sense, refers to an integrated aggregate of more than one unit. Integration testing identifies problems that occur when units are combined. By using a test plan that requires you to test each unit and ensure the viability of each before combining units, you know that any errors discovered when combining units are likely related to the interface between units. This method reduces the number of possibilities to a far simpler level of analysis.

Progressively larger groups of tested software components corresponding to elements of the architectural design are integrated and tested until the software works as a system.

I have performed integration testing whenever I have combined two modules together. When two modules are combined I have checked whether the functionality works correctly or not through integration testing

USER ACCEPTANCE TESTING

Acceptance Testing is a level of the software testing process where a system is tested for acceptability. User Acceptance testing is the software testing process where system tested for acceptability & validates the end to end business flow. Such type of testing executed by client inseparate environment & confirms whether system meets the requirements as per requirement specification or not

1. UAT is performed after System Testing is done and all or most of the major defects have been fixed. This testing is to be conducted in the final stage of Software Development Life Cycle (SDLC) prior to system being delivered to a live environment. UAT users or end users are concentrating on end to end scenarios & typically involves running a suite of tests on the completed system.

2. User Acceptance testing also known as Customer Acceptance testing (CAT), if the system is being built or developed by an external supplier. The CAT or UAT are the final confirmation from the client before the system is ready forproduction. The business customers are the primary owners of these UAT tests. These tests are created by business customers and articulated in business domain languages.

So ideally it is collaboration between business customers, business analysts, testers and developers. It consists of test suites which involve multiple test cases & each test case contains input data (if required) as well as the expected output. The resultof test case is either a pass or faiI

BETA TESTING

Beta Tests are conducted by the customers or end users at their sites. These tests are conducted in a real environment that cannot be controlled by the developer or programmer.

ALPHA TESTING

Alpha Tests are conducted at the developer's site by some potential customers. These tests are conducted in a small environment. Alpha tests are performed to identify the bugs before releasing the product to users or customers. Security testing is not checked in alpha testing.

SYSTEM IMPLEMENTATION

1. System Description
2. System Implementation

SYSTEM DESCRIPTION

A System Description is the way in which management describes to its users the very System that supports the delivery of products, solutions or services to its customers, also called system users.

SYSTEM IMPLEMENTATION

A crucial phase in the system life cycle is the successful implementation of the new system design. Implementation involves creating computer compatible files, training the operating staff, installing hardware, terminals. In the system implementation, user training is crucial for minimizing resistance to change and giving the new system a chance to prove its worth. The objectives of the system implementation is to put the system into operation while holding costs, risks and personal irritation to minimum. Once the physical system has been designed in details, the next stage is to run the design into a working system and then to monitor the operation of the system to ensure that is continue to work efficiently and the operation of the system to ensure that is continues to work efficiently and effectively. The implementation stage is often very complex and time consuming because many more people are involved than in the earlier stages.

The system implementation took place through various stages as follows

- Implantation planning
- System testing.
- System implementation.
- Change over.
- Education and training.

The implementation plan includes a description of all the activities that must occurs to implement the new system and to put it into operation. To achieve the objectives and benefits from computer-based system, it is essential for the people who will be confident of their role in the new jobs. After software is developed to meet user's requirements, users test it for acceptance.The changes over phase is used to provide adaptability for the new system.

FUTURE ENHANCEMENT

The GreenHub platform has significant potential for future enhancements, as its current design allows flexibility for adding new modules and features. Planned improvements could include real-time charging status tracking, AI-based station recommendations, and dynamic pricing models to help users make smarter charging decisions. These upgrades will enhance user convenience, reduce waiting times, and improve the overall booking and management experience for both users and charging station owners.

Additionally, the platform can be expanded with a dedicated mobile application and offline booking notifications, ensuring seamless access and usability across all devices.

Other potential future enhancements include:

- Multilingual Support: Offering the platform in multiple languages to cater to a broader user base across different regions.
- Smart Navigation Assistance: Integrating map-based navigation to guide users to the nearest charging stations with available slots.
- Community Reviews & Ratings: Allowing users to share feedback and rate charging stations to help others make informed choices.

These advancements will ensure that GreenHub remains efficient, scalable, user-friendly, and adaptable to the evolving needs of the electric vehicle ecosystem.

CONCLUSION

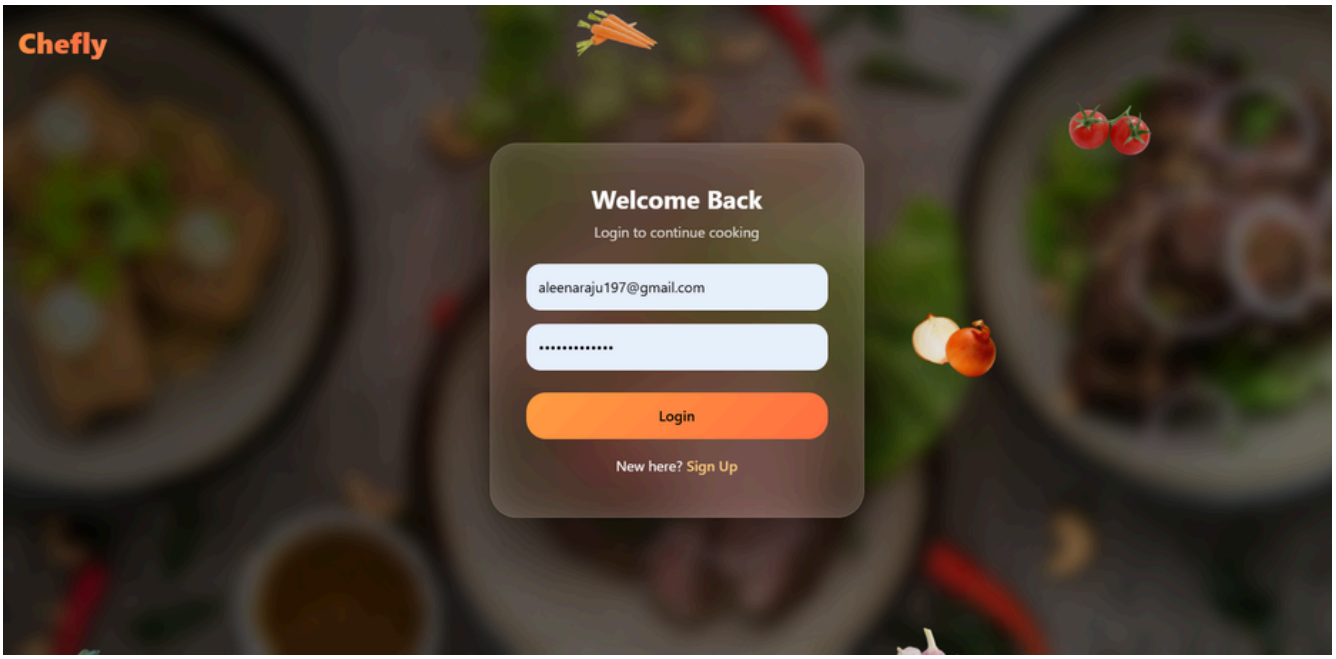
In conclusion, the Chefly project demonstrates how a digital platform can make recipe management and cooking guidance more convenient and efficient for users. The system allows users to explore a variety of recipes, manage their own recipes, and access helpful cooking information through a simple and user-friendly interface. This helps users easily find meal ideas, organize ingredients, and improve their cooking experience.

The platform also allows users to contribute their own recipes, which can be reviewed and approved by the administrator before being made available to others. This ensures that the recipes shared on the platform are reliable and useful for all users.

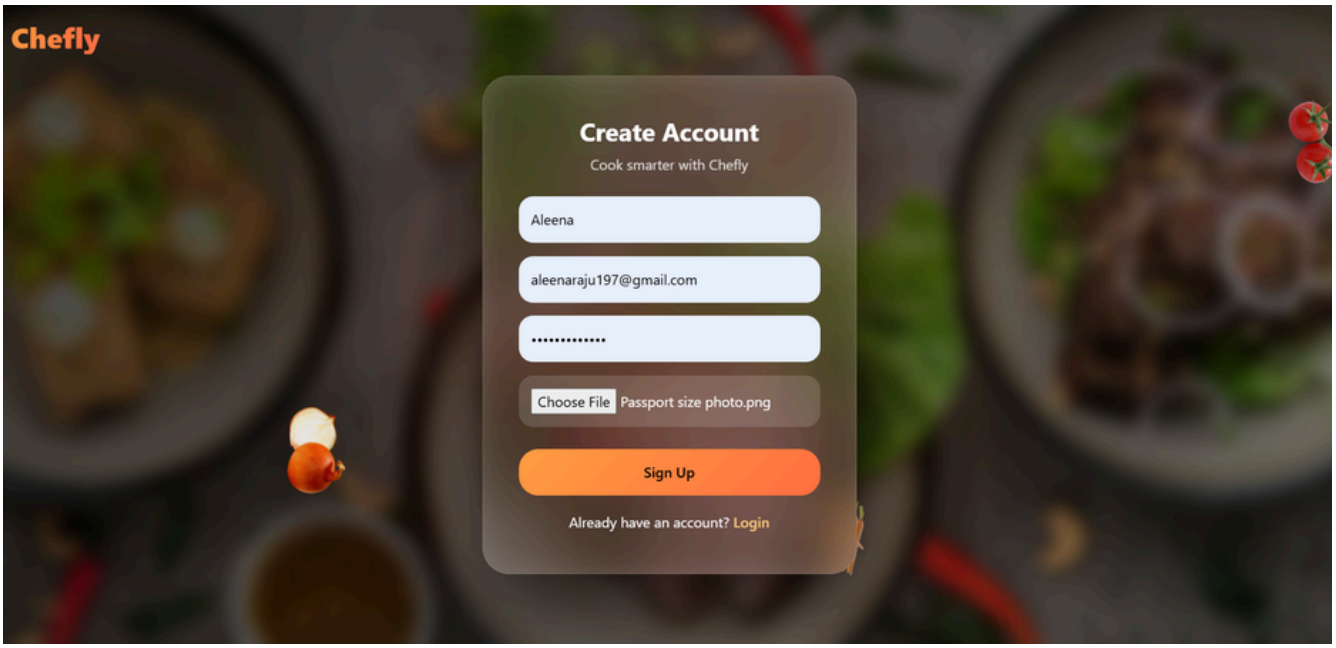
Overall, Chefly provides a modern and effective approach to managing recipes and cooking resources. It helps users discover new dishes, plan meals, and share their culinary ideas, making cooking more enjoyable and organized.

SCREENSHOTS

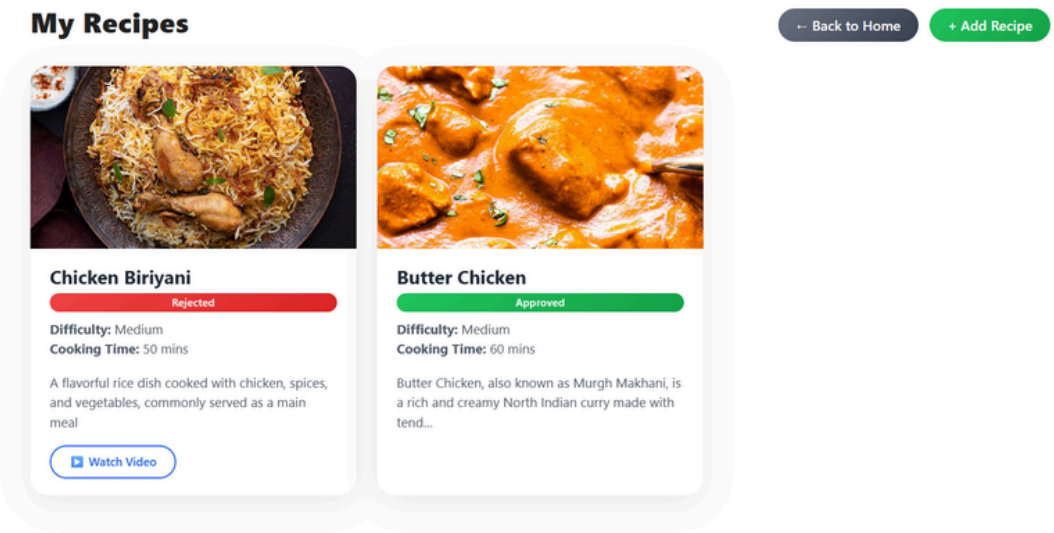
LOGIN



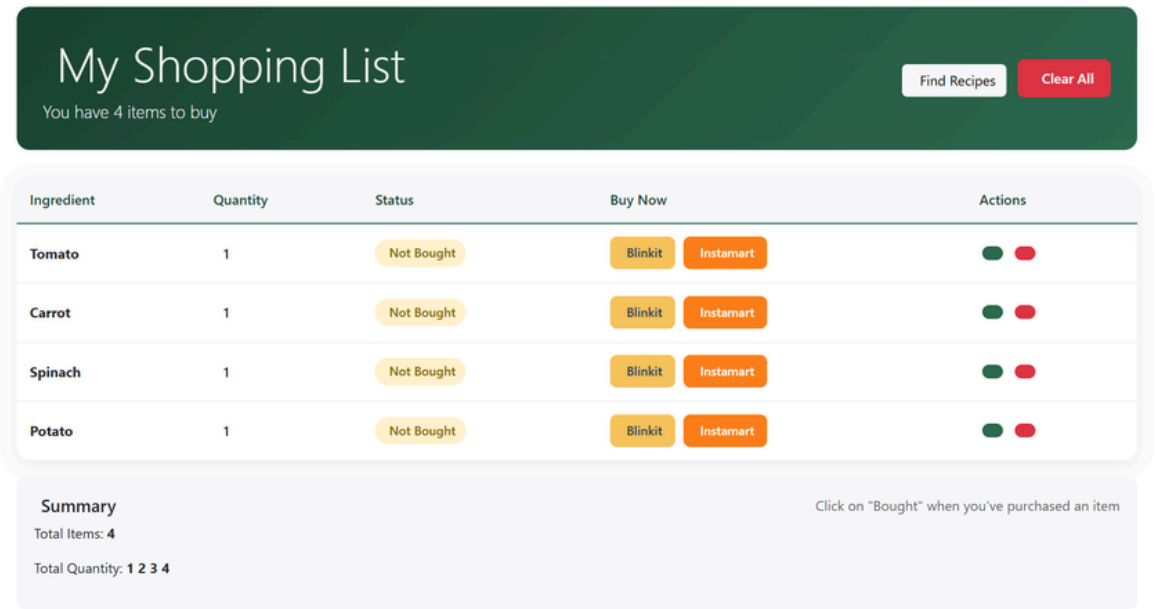
USER REGISTRATION



USER RECIPES



SHOPPING LIST



CODE

LOGIN

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Chefly | Login</title>
   <!-- Bootstrap CSS -->
   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
   <!-- Google Fonts -->
   <link href="https://fonts.googleapis.com/css2?
family=Quicksand:wght@400;500;600&display=swap" rel="stylesheet">
   <style>
      :root {
         --primary: #FFB347;   /* Light Orange */
         --secondary: #FFD580; /* Soft Yellow */
         --bg: #fffdf8;        /* Very Light Cream */
      }
 body {
         font-family: 'Quicksand', sans-serif;
         background: var(--bg);
         min-height: 100vh;
         display: flex;
         align-items: center;
         justify-content: center;
      }

 .login-card {
         background: #fff;
         border-radius: 20px;
         box-shadow: 0 20px 50px rgba(0,0,0,0.08);
         overflow: hidden;
         max-width: 900px;
         width: 100%;
         display: flex;
         flex-wrap: wrap;
      }
```

```
/* LEFT PANEL */
    .login-left {
        flex: 1 1 400px;
        padding: 50px 30px;
    }

    .login-left h2 {
        color: var(--primary);
        font-weight: 600;
        margin-bottom: 15px;
    }

    .login-left p {
        color: #555;
        margin-bottom: 30px;
    }

.form-control {
        border-radius: 12px;
        padding: 12px 15px;
        border: 1px solid #eee;
        margin-bottom: 15px;
    }

    .btn-login {
        width: 100%;
        padding: 14px;
        border-radius: 14px;
        background: linear-gradient(135deg, var(--primary), var(--secondary));
        border: none;
        color: #fff;
        font-weight: 500;
        font-size: 15px;
    }


    .btn-login:hover {
        box-shadow: 0 10px 20px rgba(255,179,71,0.3);
    }

/* RIGHT PANEL */
    .login-right {
        flex: 1 1 400px;
        background: linear-gradient(135deg, #FFF5E5, #FFEFD5)
```

```
color: #555;
        padding: 50px 30px;
        display: flex;
        flex-direction: column;
        justify-content: center;
        align-items: center;
        text-align: center;
        position: relative;
        overflow: hidden;
    }

    .login-right h2 {
        font-weight: 600;
        margin-bottom: 15px;
        color: #FFB347;
    }

    .login-right p {
        line-height: 1.6;
        margin-bottom: 25px;
    }

    .signup-link a {
        color: #FF7F50;
        font-weight: 600;
        text-decoration: underline;
    }

    /* Floating emojis */
    .emoji-floating {
        position: absolute;
        font-size: 80px;
        opacity: 0.15;
        animation: float 6s ease-in-out infinite;
        z-index: 0;
    }

  .emoji1 { top: 20px; left: 10%; animation-delay: 0s; }
        .emoji2 { top: 100px; left: 50%; animation-delay: 1s; }
        .emoji3 { top: 200px; left: 30%; animation-delay: 2s; }
        .emoji4 { top: 300px; left: 70%; animation-delay: 3s; }
```

```
        @keyframes float {
            0%, 100% { transform: translateY(0px); }
            50% { transform: translateY(-15px); }
        }
    /* Toast */
        #toast {
            position: fixed;
            top: 20px;
            right: 20px;
            background-color: #28a745; /* green for success */
            color: white;
            padding: 15px 20px;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.2);
            display: none;
            z-index: 1000;
        }
  @media (max-width: 768px) {
            .login-card {
                flex-direction: column;
            }
            .emoji-floating {
                font-size: 50px;
            }
        }

 </style>
</head>
<body>

<div class="login-card">
    <!-- LEFT PANEL: Login Form -->
    <div class="login-left">
        <h2>Welcome Back 👋</h2>
        <p>Sign in to continue cooking smarter with Chefly.</p>

       <form method="POST" action="{% url 'login' %}">

{% csrf_token %}
            <input type="email" name="email" class="form-control" placeholder="Email"
required>
            <input type="password" name="password" class="form-control"
placeholder="Password" required>
            <button type="submit" class="btn-login">Login</button>
```

```
 </form>
   </div>

   <!-- RIGHT PANEL: Info and Signup -->
   <div class="login-right">
      <h2>Hello, Chef 🍳</h2>
      <p>Discover personalized recipes, manage ingredients, and cook with
confidence using Chefly.</p>
      <div class="signup-link">
         Don't have an account? <a href="{% url 'signup' %}">Sign Up</a>
      </div>

      <!-- Floating emojis -->
      <div class="emoji-floating emoji1">🍳</div>
      <div class="emoji-floating emoji2">🥄</div>
      <div class="emoji-floating emoji3">🍅</div>
      <div class="emoji-floating emoji4">🥕</div>
   </div>
</div>

<!-- TOAST MESSAGES -->
{% if messages %}
<div id="toast">
   {% for message in messages %}
      {{ message }}
   {% endfor %}
</div>

<script>
document.addEventListener("DOMContentLoaded", function() {
   var toast = document.getElementById("toast");
   if(toast){
      toast.style.display = "block"; // show toast
      setTimeout(function() {
         toast.style.display = "none"; // hide after 2 seconds
      }, 2000);
   }
});
</script>
{% endif %}

</body>
</html>
```

SIGNUP

```
{% extends "base.html" %}
{% load static %}

{% block title %}Sign Up{% endblock %}

{% block content %}

<link href="https://fonts.googleapis.com/css2?
family=Quicksand:wght@400;500;600&display=swap" rel="stylesheet">

<style>
:root {
    --primary: #FFB347;
    --secondary: #FFD580;
}


body {
    font-family: 'Quicksand', sans-serif;
    background: #fffdf8;
}

.signup-card {
    max-width: 900px;
    margin: 80px auto;
    display: flex;
    border-radius: 20px;
    overflow: hidden;
    box-shadow: 0 20px 50px rgba(0,0,0,0.08);
}

.signup-left {
    flex: 1;
    padding: 50px;
}

.signup-left h2 {
    color: var(--primary);
}
.signup-left input {
    width: 100%;
    padding: 12px;
    margin-bottom: 15px;
```

```
border-radius: 12px;
    border: 1px solid #eee;
}

.btn-signup {
    width: 100%;
    padding: 14px;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    border: none;
    color: white;
    border-radius: 14px;
}

.signup-right {
    flex: 1;
    background: linear-gradient(135deg, #FFF5E5, #FFEFD5);
    padding: 50px;
    text-align: center;
}
</style>

<div class="signup-card">
    <div class="signup-left">
        <h2>Create Account 👨‍🍳</h2>
        <form method="post" enctype="multipart/form-data">
            {% csrf_token %}
            <input type="text" name="name" placeholder="Username" required>
            <input type="email" name="email" placeholder="Email" required>
            <input type="password" name="password" placeholder="Password" required>
            <input type="file" name="image">
            <button class="btn-signup">Sign Up</button>
        </form>
    </div>

    <div class="signup-right">
        <h2>Welcome Back 🍳</h2>
        <p>Already have an account?</p>
        <a href="{% url 'login' %}">Login</a>
    </div>
</div>

{% endblock %}
```